Факультет социально-экономических и компьютерных наук

Кафедра информационных технологий в бизнесе

Пермь
2026

Задание 5

Ассоциативные правила и кластеризация (методы обучения без учителя)

Датасет (Unis.csv) «IT-выпускники: траектории трудоустройства»

Контекст
Вы — аналитик в крупной IT-компании, которая ежегодно нанимает выпускников вузов на позиции разработчиков, аналитиков и специалистов по бизнес-информатике.
В последние годы компания столкнулась с двумя проблемами:
1. Высокий процент отказов от офферов среди выпускников топ-вузов.
2. Разрыв между ожидаемой и реальной зарплатой у новых сотрудников, что приводит к быстрой текучке в первый год работы.

Руководство поручило вам исследовать рынок труда IT-выпускников, чтобы:
• Понять, какие факторы действительно влияют на успешное трудоустройство и стартовую зарплату.
• На основе вашего анализа определить, какие характеристики (сочетание направления, GPA, наличия стажировки, региона, типа вуза и т.д.) наиболее сильно связаны с высокой стартовой зарплатой и быстрым трудоустройством.
Построить профиль «идеального кандидата» — описать его портрет. Определить, какие факторы являются обязательными, а какие желательными. Оценить, насколько этот профиль отличается для разработчиков, аналитиков и бизнес-информатиков.

Структура датасета (датасет содержит 27 переменных, которые можно разделить на несколько групп):

1. Идентификаторы и базовые характеристики
— id — уникальный номер выпускника
— year_graduation — год выпуска (2018–2025)
— region — федеральный округ
— city_type — тип города (мегаполис / областной центр / малый город)
— university_type — тип вуза (национальный исследовательский / классический / отраслевой)

2. Образовательные характеристики
— it_direction — IT-направление (разработка / аналитика / бизнес-информатика)
— specialization — узкая специализация внутри направления
— degree — уровень образования (бакалавр / специалист / магистр)
— exam_math, exam_russian, exam_informatics — баллы ЕГЭ по предметам
— exam_total — суммарный балл ЕГЭ
— gpa — средний балл диплома (GPA, от 3.0 до 5.0)
— gpa_relative — место в рейтинге группы (в процентах)

3. Опыт и навыки
— has_github — наличие публичного портфолио/GitHub (1 — да, 0 — нет)
— github_quality — качество портфолио (базовое / продвинутое)
— internship — наличие стажировки в IT (1 — да, 0 — нет)
— internship_company_type — тип компании стажировки (BigTech / стартап / аутсорс)
— internship_field — соответствие стажировки специальности (по_специальности / не_по_специальности)

4. Карьерные траектории
— employment_status — статус через 6 месяцев после выпуска (работает по специальности / работает не по специальности / учится дальше / не работает)
— first_salary — первая зарплата (руб/мес)
— salary_in_field — зарплата выше средней по региону? (1 — да, 0 — нет)
— job_search_time — время поиска работы (месяцев, от 0 до 18)
— has_job_offer_before_graduation — предложение работы до выпуска (1 — да, 0 — нет)
— expectation_salary — ожидаемая зарплата до выпуска (руб/мес)

5. Демография
— gender — пол (М / Ж)
— age_at_graduation — возраст на момент выпуска

Датасет (Autos.csv) «Данные о стоимости автомобилей с пробегом»

Контекст
Вы — аналитик в компании по продаже подержанных автомобилей, которая работает в нескольких штатах и ежегодно закупает тысячи автомобилей с аукционов, у дилеров и частных лиц. В последние годы компания столкнулась с двумя проблемами:

1.Высокая доля убыточных сделок — часть автомобилей закупается по завышенной цене и потом не окупается при перепродаже.

2.Разрыв между ожидаемой и реальной ценой продажи — менеджеры неверно оценивают рыночную стоимость автомобиля, что приводит к затянувшимся срокам продажи и снижению маржи.

Руководство поручило вам исследовать рынок подержанных автомобилей, чтобы:
• понять, какие факторы действительно влияют на **цену** и **скорость продажи** автомобиля.
• определить, какие характеристики (сочетание марки, модели, года выпуска, пробега, региона, состояния и т.д.) наиболее сильно связаны с высокой ценой и быстрой реализацией.
• построить **профиль «идеального автомобиля» для закупки** — описать его портрет. Определить, какие факторы являются обязательными, а какие — желательными.
• оценить, насколько этот профиль отличается для разных сегментов: **бюджетных**, **средних** и **премиальных** автомобилей.

Структура набора данных :

Id – уникальный номер записи
Price – цена
Year – год выпуска
Mileage – пробег в милях
City – город продажи
State – штат
Vin – номер автомобиля
Make – производитель
Model - модель

Шаг1: Базовая часть (полное выполнение — 8 баллов)

1.1) Продолжите работу с файлами и результатами полученными при выполнении третьего задания или используйте один из предложенных файлов (если используете свои файлы данных то можете продолжить работать с ними если они подходят для выполнения задания).
1.2) Выполните предобработку данных — стандартизацию признаков (приведение к нулевому среднему и единичной дисперсии). Реализуйте метод k-Means для числа k от 2 до 7. Для выбора числа кластеров используйте метод локтя.
1.3) Визуализируйте результаты кластеризации.
1.4) Исследуйте возможность выделения особых случаев в данных используя метод Apriori. Для этого постройте как минимум 3 набора правил, на основании метрик Support, Lift и Confidence и подберите пороговые значения для этих метрик.
1.5) Проанализируйте как меняются ТОП-5 рекомендаций в зависимости от метрики? Какие рекомендации кажутся наиболее логичными? Почему? Какие рекомендации выглядят странно? С чем это может быть связано?

Дополнительная к базовой часть :

Шаг 2: +1 балл
2.1) Проделать все операции первого шага на своем файле данных (рекомендуется использовать файл данных содержащий историю транзакций (чеки, заказы) или взаимодействий (просмотры, покупки)).
2.2) Разбейте один из признаков на классы например по цене для файла Autos.cdv (бюджет/средний/премиум). Используя выделенные правила присвойте выделяемым записям соответствующие классы. Постройте модели классификации на оставшихся данных. Сравните полученные результаты с результатами получаемыми без такой предварительной подготовки данных. Для сравнения результатов используйте метрики из предыдущей работы.

Шаг 3: еще +1 балл (итого +2 балла к базовой части)
3.2) Постройте на основе данных матрицу инцидентности (рёбра можно создать на основе ассоциативных правил (например, если между есть правило с Lift > 1, либо на основе Confidence). Заполните матрицу (присвойте рёбрам веса) так чтобы значения ячеек/веса ребер принимали значения Lift или Confidence соответствующего правила и примените алгоритм PageRank к этому графу. Например для файла Autos это может быть граф, где вершины — марки, модели, штаты; рёбра — совместная встречаемость.
3.3) Сделайте выводы о полученных результатах. Поясните в чем сильные и слабые стороны каждого используемого подхода в контексте ваших данных?

Стандартизация признаков

1) Загрузим данные

2) Разделим данные на признаки (***X***) и целевую переменную (***y***). Хотя в кластеризации мы не используем ***y*** для обучения, он нам понадобится для оценки

3) Применим стандартизацию к признакам.

Стандартизация выполняется по формуле:

$$z = \frac{(x-u)}{s},$$

где ***u*** — среднее значение, ***s*** — стандартное отклонение.

k-Means алгоритм

(*k*-Means=метод k средних= метод динамических ядер)

Метод k средних (k-means) — это один из самых распространенных итерационных алгоритмов кластеризации, целью которого является разбиение данных на ***k*** кластеров так, чтобы объекты внутри одного кластера были максимально похожи друг на друга, а объекты из разных кластеров — максимально различны.

Работа алгоритма:

Шаг 1: Инициализация. Определить начальные позиции центроидов (центров) для ***k*** кластеров. Это может быть сделано случайным выбором ***k*** объектов из набора данных, с опорой на дополнительную информацию или с помощью более сложных эвристик.

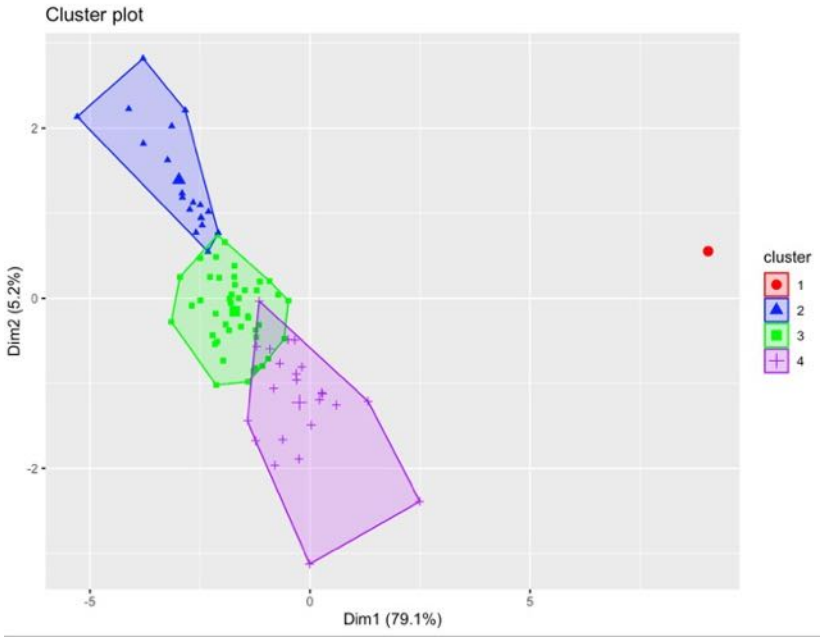
Шаг 2: Итеративное уточнение. Выполнять до тех пор, пока назначение объектов к кластерам не перестанет изменяться (или изменения будут меньше заданного порога):

1) Фаза назначения: Для каждого объекта вычисляется расстояние до всех ***k*** центроидов. Объект относится к тому кластеру, чей центроид является для него **ближайшим**.

2) Фаза обновления: После того как все объекты приписаны к кластерам, позиция центроида для каждого кластера пересчитывается. Новый центроид вычисляется как среднее арифметическое (центр масс) всех объектов, входящих в этот кластер на текущей итерации.

k-means алгоритм (сравнение)

```
#k-means
library(caret)
library(ggpubr)
library(factoextra)
res.km2017 <- kmeans(scale(trainData2017), 4, nstart = 25)
# K-means clusters showing the group of each individuals
res.km2017$cluster
fviz_cluster(res.km2017, data = trainData2019,
        palette = c("red", "blue", "green","purple"),
        geom = «point", ellipse.type = "convex")
```




```
> res.km2019$cluster
 30  69  81  22  34  45   1   2  66  43  64  85  50  29  38  65  39  72   5  40  24  31   6  78
 3   3   4   3   3   3   2   3   4   4   2   3   3   2   4   4   4   4   3   4   2   4   2   4
 41  19  73  80  68  61  52  20   7  33  71  59   8  25   9  82  46  47  18  10  26  21  53  42
 4   3   3   3   4   2   4   3   3   3   3   4   4   2   3   3   3   3   2   2   3   4   3   4
 27  74  75  11  54  55  51  79  28  36  12  77  83  56  57  13  44  60  14  48  76  16  67  15
 3   2   3   3   3   3   2   3   3   2   3   4   2   2   3   3   3   3   3   3   2   2   4   3
 63  49  58   3  23  35   4  62  17  84  70  32  37  86  87  88  89  90  91  92  93  94  95  96
 2   3   3   3   3   4   3   2   2   4   4   4   3   1   1   1   1   1   1   1   1   1   1   1
```

Метод локтя

Шаг 1: Выполняем кластеризацию (например, методом k-Means) для разного количества кластеров (k), обычно от 1 до 10 или чуть больше .

Шаг 2: **Расчитываем д**ля каждого значения k значение WCSS. С увеличением k дисперсия внутри кластеров всегда будет снижаться (в пределе, когда k равно числу точек, WCSS = 0), но это не означает улучшения качества кластеризации .

Шаг 3: Строим график на котором оси X количество кластеров (k), по оси Y — значения WCSS .

Шаг 4: На графике ищем точка, где снижение WCSS резко замедляется. Эта точка напоминает изгиб локтя . Количество кластеров, соответствующее этой точке, обычно считается оптимальным.

WCSS (Within-Cluster Sum of Squares) — это сумма квадратов расстояний от каждой точки данных до центра её кластера. По-другому эту метрику называют **инерцией** (inertia) или **внутрикластерной суммой квадратов**.

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

k – число кластеров
C_i - множество точек отнесенных к i-му кластеру
μ_i - центроид i-го кластера
x – точка данных

Ассоциативные правила (Apriori)

Можно описать как «Кто купил X, также купил Y».

Support (поддержка) - показатель популярности/частотности (показывает, как часто набор встречается во всех транзакциях)

```
Support(X) = (Количество транзакций, содержащих X) / (Общее количество транзакций)
```

Пример: Если набор {Молоко, Хлеб} встречается в 3 транзакциях из 10, то Support({Молоко, Хлеб}) = 3 / 10 = 0.3 (или 30%)

Confidence (достоверность) - как часто наше правило срабатывает для всего датасета (условная вероятность того, что в транзакции, где есть X, также будет и Y)

```
Confidence(X -> Y) = Support(X u Y) / Support(X)
```

Может вводить в заблуждение, если товар Y и так очень популярен. Например, если хлеб есть в 80% всех чеков (Support(Y) = 0.8), то даже случайное срабатывание правила даст высокую достоверность. Для исключения этого используется Lift (повышение).

Ассоциативные правила (Apriori)

Lift (повышение) - показывает, насколько X и Y зависят друг от друга (показывает, насколько наличие X *увеличивает* вероятность наличия Y по сравнению с базовой (безусловной) вероятностью Y —мера *зависимости* между X и Y).

```
Lift(X -> Y) = Confidence(X -> Y) / Support(Y)
```

Эквивалентная формула: `Lift(X -> Y) = Support(X ∪ Y) / (Support(X) * Support(Y))`

Lift = 1: X и Y независимы. Наличие X не влияет на вероятность Y.
Lift > 1: Положительная зависимость. Наличие X *увеличивает* вероятность Y. Чем больше лифт, тем сильнее связь. Это хорошие, полезные правила.
Lift < 1: Отрицательная зависимость. Наличие X *уменьшает* вероятность Y. Иногда это тоже полезно (например, для сегментации).

Coverage (покрытие) - показывает, насколько широко "применимо" правило. Coverage измеряет долю транзакций, в которых условие правила (левая часть, антецедент) встречается.

```
Coverage(X -> Y) = Support(X)
```

или `Coverage(X -> Y) = (Количество транзакций, содержащих X) / (Общее количество транзакций)`

Высокий Coverage (> 0.5): Правило применимо к большой части датасета. Условие встречается часто.
Низкий Coverage (< 0.1): Правило узкоспециализированно, применимо только к небольшой части данных.

Ассоциативные правила (Apriori)

Примеры правил:

lhs	rhs	support	confidence	coverage	lift	count
[1]	{X3=[0,70.2),X18=[0,5.54)} => {X2=[0,67.9)}	0.2	1	0.2	3.125000	20
[2]	{X4=[0,38.8),X6=[0,1)} => {X5=[0,28.3)}	0.2	1	0.2	3.030303	20
[3]	{X2=[0,67.9),X13=[0,51.8)} => {X7=[0,49.1)}	0.2	1	0.2	3.030303	20
[4]	{X2=[0,67.9),X7=[0,49.1)} => {X13=[0,51.8)}	0.2	1	0.2	3.030303	20
[5]	{X5=[0,28.3),X10=[0,41.9)} => {X12=[0,15.3)}	0.2	1	0.2	3.030303	20

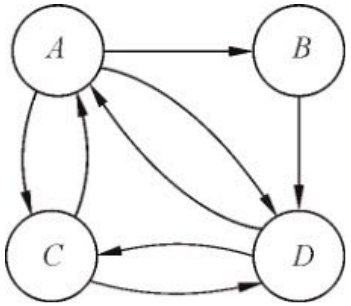
Алгоритм ссылочного ранжирования (PageRank алгоритм)

Связи между страницами или факторами могут быть представлены в виде графа. Узел представляет собой веб-страницу или фактор, а стрелка от *A* на страницу *B* означает, что на странице *A* есть ссылка на страницу *B*. Количество исходящих ссылок является важным параметром.

Обозначим через *L(p)* количество исходящих ссылок на странице *p*.

Пусть *N* — число страниц. Запишем матрицу $A(N\times N)$, значения элементов которой будем определять как $a_{i,j} = L(j)$, если есть связь между *j* и *i*, 0 в противном случае.

Пусть имеется четыре страницы. Страница *A* содержит ссылку на страницу *B*, ссылку на страницу *C* и ссылку на страницу *D*. Страница *B* содержит одну-единственную ссылку на страницу *D*. Страница *C* указывает на страницы *A* и *D*, а страница *D* указывает на страницы *A* и *C*. Они представлены в следующими выражениями: L(A)=3; L(B)=1; L(C)=2; L(D)=2.



Для рассматриваемого примера получим следующую матрицу:

$$\begin{pmatrix} 0 & 0 & 1/2 & 1/2 \\ 1/3 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1 & 1/2 & 0 \end{pmatrix}.$$

Сумма значений в каждой колонке полученной матрицы равна 1.

Алгоритм ссылочного ранжирования (PageRank алгоритм)

Упрощенно PageRank-алгоритм можно описать следующим образом — инициализируем x как столбец $N \times 1$ с неотрицательными значениями и последовательно заменяем x на произведение $A \cdot x$ до сходимости (до тех пор, пока значения не перестанут изменяться или их изменение будет меньше некоторой величины погрешности ε).

Вектор x называют PageRank-вектором. Инициализация вектора обычно осуществляется равными значениями. Далее представим себе веб-серфера, который щелкает по ссылкам случайным образом. Если на странице есть k ссылок, то он случайно выбирает одну и переходит на выбранную страницу. Через достаточно продолжительное время N компонент PageRank-вектора будут пропорциональны количеству раз, которое серфер посетил N страниц.

Для рассмотренного примера вектор будет состоять из четырех компонент, которые инициализируем единичными значениями:

$$\mathbf{x} = \begin{pmatrix} x_A \\ x_B \\ x_C \\ x_D \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

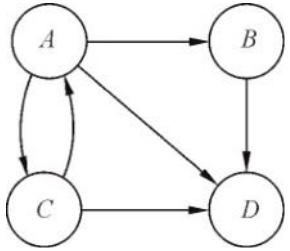
Изменение значения PageRank-вектора можно проиллюстрировать

Номер итерации	x_A	x_B	x_C	x_D
0	1	1	1	1
1	1	0.3333	0.8333	1.8333
2	1.3333	0.3333	1.25	1.0833
3	1.667	0.4444	0.9861	1.4028
4	1.1944	0.3889	1.0903	1.3264
5	1.2083	0.3981	1.0613	1.3322
6	1.1968	0.4028	1.0689	1.3316
7	1.2002	0.3989	1.0647	1.3361

Видно, что значения в приведенном примере быстро сходятся, и максимальное значение имеет узел *D* с наибольшим количеством входящих ссылок.

Алгоритм ссылочного ранжирования (PageRank алгоритм)

Возможна ситуация, когда отдельные страницы вообще не имеют исходящих ссылок



Ассоциированная с графом матрица A будет иметь вид

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 1/2 & 0 \\ 1/3 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 0 \\ 1/3 & 1 & 1/2 & 0 \end{pmatrix}$$

В данном случае последний столбец принимает нулевые значения, в результате чего алгоритм сходится к нулевым значениям, что не верно.

Номер итерации	x_A	x_B	x_C	x_D
0	1	1	1	1
1	0.5	0.3333	0.8333	1.8333
2	0.1667	0.1667	0.1667	0.6667
3	0.0833	0.0556	0.0556	0.3056
4	0.0278	0.0278	0.0278	0.1111
5	0.0139	0.0093	0.0098	0.0509

Для того чтобы алгоритм работал, значения таких "висящих" узлов будем заменять значением $1/N$ при построении матрицы A.
Тогда для рассматриваемого примера A примет вид:

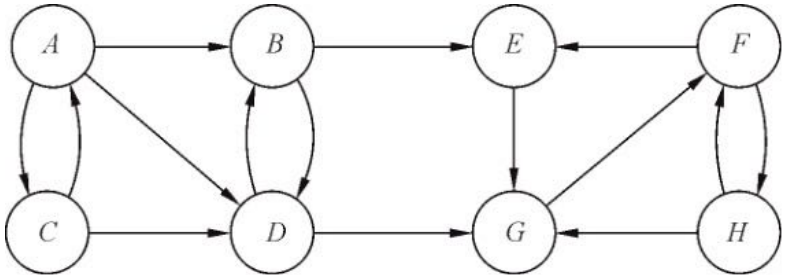
$$\overline{\mathbf{A}} = \begin{pmatrix} 0 & 0 & 1/2 & 1/4 \\ 1/3 & 0 & 0 & 1/4 \\ 1/3 & 0 & 0 & 1/4 \\ 1/3 & 1 & 1/2 & 1/4 \end{pmatrix}$$

и алгоритм снова сходится и дает верные значения

Номер итерации	x_A	x_B	x_C	x_D
0	1	1	1	1
1	0.75	0.5833	0.5833	2.0833
2	0.8125	0.7708	0.7708	1.6458
3	0.7969	0.6823	0.6823	1.8385
4	0.8008	0.7253	0.7253	1.7487
5	0.7998	0.7041	0.7041	1.7920

Алгоритм ссылочного ранжирования (PageRank алгоритм)

При увеличении числа узлов может получиться, что матрица будет содержать большое количество нулей



$$\bar{\mathbf{A}} = \begin{pmatrix} 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1/2 & 1/2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 0 & 0 & 1/2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1/2 \\ 0 & 0 & 0 & 1/2 & 1 & 0 & 0 & 1/2 \\ 0 & 0 & 0 & 0 & 0 & 1/2 & 0 & 0 \end{pmatrix}.$$

Если начать работу алгоритма на этих данных, то мы получаем

Номер итерации	x_A	x_B	x_C	x_D	x_E	x_F	x_G	x_H
0	0.125	0.125	0.125	0.125	0.125	0.125	0.125	0.125
1	0.0625	0.1042	0.0417	0.1667	0.125	0.1875	0.25	0.0625
2	0.0208	0.1042	0.0208	0.0937	0.1458	0.2813	0.2396	0.0938
3	0.0104	0.0538	0.0069	0.0697	0.1927	0.2865	0.2396	0.1406
4	0.0035	0.0382	0.0035	0.0339	0.1701	0.3099	0.2977	0.1432
5	0.0017	0.0181	0.0012	0.0220	0.1740	0.3694	0.2587	0.1549
6	0.0006	0.0116	0.0006	0.0102	0.1937	0.3362	0.2625	0.1847
7	0.0003	0.0053	0.0002	0.0063	0.1739	0.3549	0.2912	0.1681
8	0.0001	0.0032	0.0001	0.0028	0.1801	0.3752	0.2610	0.1774

Из расчета видно, что рейтинг узлов *A* и *D* опускается до нуля, а это не верно. Это значит, что при работе с разряженными матрицами алгоритм не работает.

Алгоритм ссылочного ранжирования (PageRank алгоритм)

Для того чтобы скорректировать работу алгоритма, будем использовать матрицу M, которая получается следующим образом:

$$\mathbf{M} = d \cdot \bar{\mathbf{A}} + \frac{1-d}{N} \cdot \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & 1 & 1 & \cdots & 1 \\ 1 & 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 1 \end{pmatrix}$$

где *d* принимает значение между 0 и 1 и называется коэффициентом или постоянной демпфирования (ее значение в поисковой системе Google по умолчанию равно 0.85).

Со значением d=0.85 для рассматриваемого примера получим:

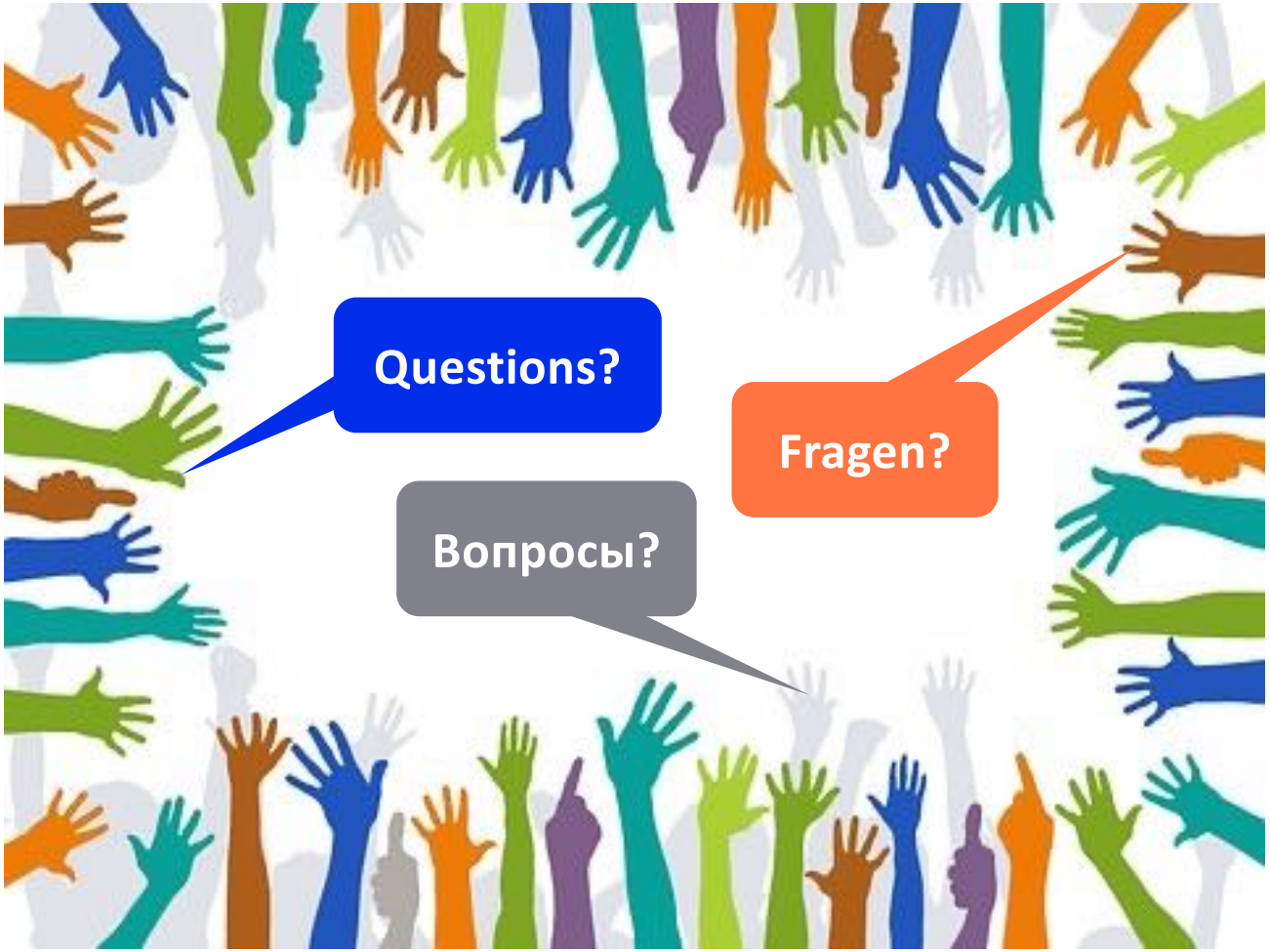
$$\mathbf{M} = \begin{pmatrix} 0.0187 & 0.0187 & 0.4437 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 \\ 0.3021 & 0.0187 & 0.0187 & 0.4437 & 0.0187 & 0.0187 & 0.0187 & 0.0187 \\ 0.3021 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 \\ 0.3021 & 0.4437 & 0.4437 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 \\ 0.0187 & 0.4437 & 0.0187 & 0.0187 & 0.0187 & 0.4437 & 0.0187 & 0.0187 \\ 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.8688 & 0.4437 \\ 0.0187 & 0.0187 & 0.0187 & 0.4437 & 0.8688 & 0.0187 & 0.0187 & 0.4437 \\ 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.0187 & 0.4437 & 0.0187 & 0.0187 \end{pmatrix}.$$

Используя новую матрицу, получим следующие значения

Номер итерации	x_A	x_B	x_C	x_D	x_E	x_F	x_G	x_H
0	0.125	0.125	0.125	0.125	0.125	0.125	0.125	0.125
1	0.0719	0.1073	0.0542	0.1604	0.1250	0.1781	0.2313	0.0719
2	0.0418	0.1073	0.0391	0.1077	0.1401	0.2459	0.2237	0.0945
3	0.0354	0.0764	0.0306	0.0928	0.1688	0.2491	0.2237	0.1232
4	0.0317	0.0682	0.0288	0.0742	0.1571	0.2613	0.2541	0.1246
5	0.0310	0.0593	0.0277	0.0690	0.1588	0.2877	0.2368	0.1298
6	0.0305	0.0568	0.0275	0.0645	0.1662	0.2752	0.2382	0.1410
7	0.0304	0.0548	0.0274	0.0633	0.1598	0.2811	0.2474	0.1357
8	0.0304	0.0543	0.0274	0.0623	0.1615	0.2867	0.2392	0.1382

Из теоремы Фробениуса–Перрона следует, что в данном виде алгоритм всегда будет сходиться.

Теорема Фробениуса — Перрона — *теорема о наибольшем собственном значении вещественной квадратной матрицы с положительными компонентами.*


Questions?
Fragen?
Вопросы?